

MÔ TẢ ĐỀ TÀI ĐỒ ÁN TỐT NGHIỆP

Thành viên	Mai Khoa Bách Nguyễn Minh Đức
GVHD	ThS. Lê Văn Minh
Tên đề tài	Xây dựng hệ thống phát hiện bất thường và hỗ trợ phân tích nguyên nhân sự cố trong kiến trúc microservice dựa trên dữ liệu observability và học máy

1. Bối cảnh và lý do chọn đề tài

Trong kiến trúc microservice, một yêu cầu của người dùng có thể đi qua nhiều service, cơ sở dữ liệu, cache và message queue. Khi một thành phần xảy ra sự cố, biểu hiện lỗi thường lan truyền sang các service khác, khiến developer khó xác định đâu là nguyên nhân ban đầu và đâu chỉ là triệu chứng. Việc kiểm tra thủ công metrics, distributed traces và logs trên nhiều nguồn vừa tốn thời gian vừa dễ bỏ sót mối liên hệ giữa các thành phần.

Nhóm dự kiến xây dựng một hệ thống hỗ trợ phát hiện bất thường và khoanh vùng nguyên nhân sự cố cho môi trường microservice. Để chủ động về dữ liệu, workload và ground truth, nhóm đồng thời xây dựng một backend microservice theo miền nghiệp vụ LMS ở phạm vi thu gọn làm môi trường thử nghiệm. LMS không phải sản phẩm độc lập của đồ án mà đóng vai trò System Under Test cho phân observability, AI và Root Cause Analysis.

2. Mục tiêu của đề tài

Mục tiêu tổng quát là xây dựng một hệ thống kỹ thuật hoàn chỉnh gồm microservice testbed, observability pipeline và nền tảng phân tích sự cố bằng AI/ML.

Các mục tiêu chính gồm:

- Xây dựng một backend microservice đủ phức tạp để tạo ra các quan hệ phụ thuộc thực tế giữa nhiều service.
- Thu thập metrics, distributed traces và logs bằng OpenTelemetry.
- Xây dựng pipeline xử lý và trích xuất đặc trưng từ telemetry.
- Sử dụng các phương pháp học máy không giám sát để phát hiện hành vi bất thường của service.
- Xây dựng cơ chế phân tích dependency và thứ tự xuất hiện bất thường để xếp hạng các root-cause candidates.
- Cung cấp incident timeline và các metric, trace, log làm bằng chứng hỗ trợ developer/SRE điều tra sự cố.

- Chủ động tạo các fault có kiểm soát để xây ground truth và đánh giá hệ thống bằng các chỉ số định lượng.

3. Phạm vi backend microservice làm testbed

Nhóm dự kiến sử dụng miền nghiệp vụ Learning Management System (LMS) nhưng chỉ triển khai các chức năng cần thiết để tạo một testbed có ý nghĩa cho bài toán chính. Hệ thống có thể gồm các service như API Gateway, Auth, Course, Enrollment, Assignment, Submission, Grading và Notification.

Testbed dự kiến sử dụng PostgreSQL cho dữ liệu nghiệp vụ, Redis cho cache, RabbitMQ cho giao tiếp bất đồng bộ và có thể sử dụng object storage hoặc storage mock cho bài nộp. Một số luồng chính gồm đăng nhập, xem khóa học, đăng ký học, nộp bài, chấm điểm và gửi thông báo.

Phần backend được thiết kế để tạo ra nhiều dạng dependency: service-to-service call, database dependency, cache dependency và message queue. Nhóm không đặt mục tiêu xây dựng LMS đầy đủ với các chức năng như chat, forum, video streaming, AI tutor hoặc hệ thống khuyến nghị vì những phần này không phục vụ trực tiếp cho bài toán anomaly detection và RCA.

4. Observability và dữ liệu đầu vào

Các service được instrument bằng OpenTelemetry ngay từ giai đoạn phát triển. Hệ thống dự kiến thu thập ba loại telemetry chính:

- Metrics: request rate, error rate, latency, CPU, memory và một số metric đặc thù của service.
- Distributed traces: đường đi của request qua các service, thời gian xử lý từng span và quan hệ caller-callee.
- Logs: structured logs có service identity và trace/span correlation để truy vết các lỗi liên quan.

OpenTelemetry Collector được dùng để thu nhận và chuyển tiếp telemetry. Nhóm dự kiến sử dụng Prometheus cho metrics, Grafana Tempo cho traces và Loki cho logs. Các công cụ này chủ yếu đóng vai trò lưu trữ và quan sát dữ liệu gốc; phần anomaly detection và RCA sẽ do nhóm tự xây dựng.

5. Hướng triển khai AI/ML

AI của đề tài không phải một tính năng chatbot hoặc gọi API đến LLM. Phần AI nằm trực tiếp trong pipeline phân tích telemetry và giải quyết bài toán cho developer/SRE.

Trước hết, dữ liệu telemetry theo từng service sẽ được chuẩn hóa theo các cửa sổ thời gian và chuyển thành các đặc trưng như request rate, error rate, p95 latency, CPU, memory, downstream latency, trace error rate và log error rate.

Nhóm dự kiến xây dựng các baseline thống kê như threshold hoặc robust z-score, sau đó so sánh với ít nhất một phương pháp học máy không giám sát như Isolation Forest. Hướng unsupervised được ưu tiên vì trong thực tế dữ liệu incident có nhãn thường ít, trong khi dữ liệu hoạt động bình thường có thể thu thập dễ hơn.

Sau khi phát hiện incident, hệ thống xây dựng đồ thị phụ thuộc động giữa các service từ distributed traces. Mỗi service được đánh giá dựa trên nhiều loại bằng chứng, dự kiến gồm mức độ bất thường, thời điểm bắt đầu bất thường, quan hệ với các service bị ảnh hưởng và mức độ suy giảm trên dependency. Từ đó hệ thống trả về danh sách root-cause candidates theo thứ hạng thay vì khẳng định tuyệt đối một nguyên nhân.

Ví dụ, nếu database của Submission Service bị chậm, sự cố có thể lần lượt làm Submission, API Gateway và các luồng liên quan tăng latency. Hệ thống cần nhận ra rằng service có anomaly score cao nhất chưa chắc là root cause; thông tin dependency và temporal propagation sẽ được dùng để phân biệt nguyên nhân ban đầu với triệu chứng lan truyền.

Nếu tiến độ cho phép, nhóm có thể nghiên cứu bổ sung change-point detection, log-template features hoặc một số phương pháp causal/statistical analysis trên tập candidate nhỏ. Các phần nâng cao này không thuộc phạm vi bắt buộc của MVP.

6. Tạo dữ liệu và đánh giá

Để có ground truth rõ ràng, nhóm sẽ tự động sinh workload và inject fault vào testbed. Một số fault dự kiến gồm CPU saturation, network/dependency delay, service error, database latency, service crash, cache slowdown hoặc queue backlog.

Mỗi experiment sẽ lưu loại fault, service/dependency mục tiêu, thời gian bắt đầu và kết thúc, cường độ fault và workload profile. Nhờ đó có thể so sánh prediction của hệ thống với root cause thực tế đã biết.

Các chỉ số đánh giá chính dự kiến gồm Precision, Recall, F1-score và Detection Delay cho anomaly/incident detection; Top-1, Top-3, Mean Reciprocal Rank cho root-cause ranking; đồng thời đánh giá runtime, tài nguyên sử dụng và độ ổn định khi một phần telemetry bị thiếu hoặc giảm sampling.

Nhóm cũng dự kiến thực hiện baseline comparison và ablation study, ví dụ so sánh metrics-only với metrics + traces, hoặc RCA có/không có dependency graph và temporal information. Đây là phần quan trọng để chứng minh vai trò thực sự của từng thành phần AI thay vì chỉ demo chức năng.

7. Phương hướng triển khai hệ thống

Giai đoạn đầu nhóm phát triển testbed và observability bằng Docker Compose để giảm độ phức tạp và dễ tái lập. Sau khi pipeline ổn định, Kubernetes và Chaos Mesh có thể được bổ sung nếu cần cho fault injection ở mức hạ tầng.

Nền tảng phân tích AI/RCA dự kiến được xây dựng dưới dạng một backend riêng, có các module telemetry adapter, feature engineering, anomaly detector, incident engine, dependency graph builder, RCA engine và evaluation engine. Một dashboard đơn giản sẽ hỗ trợ xem incident, service graph, root-cause candidates, timeline và bằng chứng liên quan.

Trong toàn bộ quá trình, LMS được giữ ở phạm vi vừa đủ. Khi testbed đã có đủ service, dependency, workload và fault scenario cần thiết, nhóm sẽ đóng băng các chức năng nghiệp vụ và tập trung vào phân phân tích, thí nghiệm và đánh giá.

8. Kết quả dự kiến

Kết quả cuối cùng dự kiến gồm:

- Một backend microservice LMS thu gọn đóng vai trò testbed.
- Một observability pipeline thu thập metrics, traces và logs.
- Một hệ thống anomaly detection và root-cause candidate ranking có tích hợp AI/ML thực chất.
- Một bộ fault scenarios và dataset/ground truth phục vụ thực nghiệm.
- Backend API và dashboard hỗ trợ developer/SRE xem incident và bằng chứng.
- Báo cáo thực nghiệm so sánh các baseline, đánh giá hiệu quả, runtime và giới hạn của giải pháp.